

TechDocs

Plan de Desarrollo — Sistema de Documentación Técnica

Versión 1.0 | Equipo de Desarrollo | 2026

Tecnologías: NestJS Angular React GitHub Copilot Docker

1. Resumen Ejecutivo

Vision general del proyecto y objetivos

TechDocs es una aplicación para gestionar documentación técnica interna. El proyecto esta diseñado como un curso práctico de 9 módulos evolutivos, donde el equipo construye un sistema real mientras refuerza los temas con mayor índice de error detectados en la evaluación de competencias. El proceso comienza con una arquitectura monolítica simple y evoluciona hacia microservicios desplegados en la nube.

Atributo	Detalle
Nombre del proyecto	TechDocs — Sistema de Documentación Técnica
Arquitectura inicial	Monolito (NestJS + Angular + React)
Arquitectura final	Microservicios con Docker y CI/CD
Módulos totales	9 módulos iterativos (Fase 1: 5, Fase 2: 4)
Duración estimada	2 a 3 horas por modulo
Nivel requerido	Intermedio (conocimientos básicos previos)
Asistencia con IA	GitHub Copilot en cada modulo
Cobertura de brechas	13 temas críticos de Angular, React y NodeJS

2. Temas Débiles Identificados

Brechas detectadas por tecnología según evaluación de competencias

Los siguientes temas presentaron los mayores índices de error en la evaluación del equipo. Cada uno esta mapeado a un módulo específico del proyecto para garantizar su cobertura práctica.

Angular

Tema debil identificado	% Error	Nivel
Directivas Básicas	20%	Rookie
Pruebas Unitarias en Angular	19%	Junior Avanzado
Enrutamiento Avanzado y Guards	20%	Junior Avanzado
Pruebas de Integración y E2E	24%	Semi Senior
Mecanismos de Seguridad	27%	Semi Senior
Renderizado y detección de cambios avanzados	20%	Senior
Desarrollo de librerías personalizadas	60%	Senior
Optimización y practicas avanzadas de seguridad	30%	Senior
Integración continua y despliegue continuo	20%	Senior

React

Tema débil identificado	% Error	Nivel
Fundamentos de React	24%	Rookie
JSX y Renderizado	18%	Rookie

NodeJS

Tema débil identificado	% Error	Nivel
Performance y Profiling	14%	Senior
Testing y Calidad de Código	18%	Senior
DevOps, CI/CD y Nube	6%	Senior

3. Arquitectura Evolutiva

De monolito a microservicios paso a paso

El proyecto evoluciona en dos fases bien diferenciadas. Esto permite al equipo entender las ventajas y limitaciones de cada arquitectura con contexto real, no solo teórico.

	Fase 1 — Monolito	Fase 2 — Microservicios
Módulos	M1 a M5	M6 a M9
Backend	Un proceso NestJS único	Servicios NestJS independientes
Frontend	Angular + React embebido	Igual, consume multiples APIs
Base de datos	PostgreSQL compartida	BD por servicio (cuando aplica)
Comunicación	Llamadas directas internas	RabbitMQ / HTTP entre servicios
Despliegue	Un contenedor Docker	Docker Compose multi-servicio
Observabilidad	Logs locales	OpenTelemetry + Grafana

4. Módulos Detallados

Contenido, implementación y resultado esperado por modulo

Modulo 1 — Setup y Fundamentos

Fase 1 — Monolito | Duración: 2 horas | Asistencia: GitHub Copilot

Descripción

El equipo configura el repositorio mono repo en GitHub. Se instalan NestJS, Angular CLI y se integra React. Se activa GitHub Copilot en VS Code para todos los integrantes. El módulo construye la pantalla inicial: un listado de documentos en Angular con panel de preview en React-Angular.

Temas débiles que cubre

- Angular — Directivas Básicas (20%): *ngIf, *ngFor, directivas de atributo personalizadas.
- React — Fundamentos de React (24%): componentes, props, estado con useState.
- React — JSX y Renderizado (18%): Virtual DOM, como React decide re-renderizar.

Que se implementa

Que implementar	Como	Herramienta
Mono repo con nx o carpetas separadas	Estructura backend/ y frontend/ en GitHub	Node.js + Git
Módulo de documentos Angular	Componente lista con *ngIf y *ngFor sobre datos mock	Angular CLI + Copilot
Directiva personalizada	AttrDirectiva que cambia color al hacer hover	Angular
Panel preview React-Angular	Componente React.	React + Vite
Configuración de Copilot	Instalar extensión, probar sugerencias	VS Code + Copilot

Modulo 2 — CRUD de Documentos y Enrutamiento

Fase 1 — Monolito | Duración: 3 horas | Asistencia: GitHub Copilot

Descripción

Se construye la API REST de NestJS para gestionar documentos: crear, leer, actualizar y eliminar. Se conecta a PostgreSQL con TypeORM. En Angular se implementa enrutamiento completo con rutas de listado, detalle, edición y Guards.

Temas debiles que cubre

- Angular — Enrutamiento Avanzado y Guards (20%): CanActivate, rutas anidadas, lazy loading
- NodeJS — Performance y Profiling (14%): medir latencia con performance.now() y clinic.js

Que se implementa

Que implementar	Como	Herramienta
Entidad Manual NestJS	id, titulo, contenido, categoría, versión, fechaCreacion	NestJS + TypeORM
API REST completa	Controlador + Servicio + DTO con class-validator	NestJS + Copilot

Que implementar	Como	Herramienta
Enrutamiento Angular	RouterModule con rutas /docs, /docs/:id, /docs/:id/edit	Angular Router
Guard de autenticación básica	CanActivate verifica token en localStorage	Angular
Lazy loading	loadChildren para cargar editor solo cuando se necesita	Angular
Profiling de endpoints	Middleware que registra duración de cada request	NestJS + Node perf

Modulo 3 — Seguridad y Autenticación

Fase 1 — Monolito | Duración: 3 horas | Asistencia: GitHub Copilot

Descripción

Módulo de mayor impacto según la evaluación (27% de error). Se implementa JWT completo con Passport.js. En Angular: interceptores HTTP, sanitización de inputs, headers de seguridad y CSP básico.

Temas débiles que cubre

- Angular — Mecanismos de Seguridad (27%): interceptores, sanitización, DomSanitizer
- Angular — Practicas avanzadas de seguridad (30%): CSP, CORS, HTTPS headers

Que se implementa

Que implementar	Como	Herramienta
JWT en NestJS	Login endpoint + generación de token + JwtAuthGuard	NestJS + @nestjs/jwt
Interceptor HTTP Angular	AuthInterceptor adjunta 'Authorization: Bearer TOKEN'	Angular HttpClient
Sanitización de contenido	DomSanitizer para contenido HTML dinamico	Angular
Headers de seguridad	Helmet.js: X-Frame-Options, CSP, HSTS	NestJS + Helmet
CORS configurado	Solo el dominio del frontend puede llamar a la API	NestJS CORS
Pantalla de login	Formulario Angular conectado al endpoint de autenticación	Angular + Copilot

Modulo 4 — Pruebas Unitarias y E2E

Fase 1 — Monolito | Duración: 3 horas | Asistencia: GitHub Copilot

Descripción

El módulo más extenso de pruebas. Tests unitarios para cada servicio de NestJS y componentes Angular críticos. Pruebas E2E con Cypress que simulan un usuario real navegando TechDocs.

Temas débiles que cubre

- Angular — Pruebas Unitarias (19%): TestBed, ComponentFixture, Spectator
- Angular — Pruebas E2E (24%): Cypress, testing de flujos completos
- NodeJS — Testing y Calidad de Código (18%): Jest, mocks.

Que se implementa

Que implementar	Como	Herramienta
Tests unitarios NestJS	DocumentsService: crear, obtener, actualizar, eliminar con mocks	Jest
Tests componentes Angular	DocumentListComponent: renderiza lista, emite eventos	TestBed + Jasmine
Mock HttpClient	HttpClientTestingModule sin API real	Angular Testing
Pruebas E2E Cypress	Flujo: login → ver lista → crear documento → verificar	Cypress
Cobertura de código	Istanbul, meta mínima 70% en servicios	Jest + Istanbul
Reporte HTML	jest --coverage genera reporte visual del estado	Jest

Modulo 5 — Renderizado Avanzado y Librerías

Fase 1 — Monolito | Duración: 3 horas | Asistencia: GitHub Copilot

Descripción

El tema con mayor brecha (60% en Librerías Personalizadas). Se refactoriza el componente de preview en una librería Angular independiente publicable vía npm. Se aplica OnPush en todos los componentes y se mide el impacto con Angular DevTools.

Temas débiles que cubre

- Angular — Librerías personalizadas (60%): ng generate library, ng-packagr
- Angular — Renderizado y detección de cambios (20%): OnPush, signals, trackBy

Que se implementa

Que implementar	Como	Herramienta
Librería Angular techdocs-ui	ng generate library techdocs-ui con DocumentPreview	Angular CLI + nx
Publicación local	npm pack + instalar como dependencia local	npm

Que implementar	Como	Herramienta
ChangeDetectionStrategy.OnPush	Aplicar en DocumentListComponent y DocumentPreview	Angular
trackBy en *ngFor	Evitar re-renders al actualizar lista de documentos	Angular
Signals (Angular 17+)	Reemplazar BehaviorSubject por signal() y computed()	Angular Signals
Análisis DevTools	Ver árbol de detección de cambios	Chrome DevTools

Modulo 6 — CI/CD y DevOps

Fase 2 — Microservicios | Duración: 2 horas | Asistencia: GitHub Copilot

Descripción

Se configura el pipeline CI/CD con GitHub Actions. Cada push a main dispara: instalación de dependencias, ejecución de tests, build de producción, construcción de imagen Docker y despliegue automático.

Temas débiles que cubre

- Angular — Integración continua y despliegue (20%): pipeline Angular con Actions
- NodeJS — DevOps, CI/CD y Nube (6%, alta deuda técnica): Dockerfile, Actions, despliegue

Que se implementa

Que implementar	Como	Herramienta
Dockerfile backend	Imagen Node Alpine, multi-stage build	Docker
Dockerfile frontend	Build Angular + Nginx para estaticos	Docker + Nginx
GitHub Actions CI	Workflow que corre ng test y jest en cada PR	GitHub Actions
GitHub Actions CD	Build imagen y deploy en push a main	GitHub Actions
Variables seguras	Secrets de GitHub para DATABASE_URL, JWT_SECRET	GitHub Secrets
Badge de estado	Badge de CI visible en el README	GitHub Actions

Modulo 7 — Primer Microservicio — Búsqueda

Fase 2 — Microservicios | Duración: 3 horas | Asistencia: GitHub Copilot

Descripción

El punto de inflexión. Se extrae la funcionalidad de búsqueda del monolito y se convierte en microservicio independiente. El servicio principal se comunica vía RabbitMQ. El equipo entiende en práctica que significa separar responsabilidades.

Temas débiles que cubre

- Transversal: separación fuerza a aplicar Guards, DTOs y pruebas del servicio nuevo.
- NodeJS — Performance: el microservicio tiene su propio endpoint /metrics.

Que se implementa

Que implementar	Como	Herramienta
Nuevo proyecto search-service	Proyecto independiente en services/search/	NestJS CLI
RabbitMQ	search-service escucha mensajes 'search.query' y responde	NestJS Microservices
Actualización servicio principal	documents-service envia mensajes en lugar de buscar directo	RabbitMQ + NestJS
BD de búsqueda	search-service usa SQLite con FTS5	TypeORM / SQLite
Docker Compose multi-servicio	Levantar ambos servicios + RabbitMQ con docker-compose up	Docker Compose
Tests del nuevo servicio	Pruebas unitarias del SearchService con mocks del broker	Jest

Modulo 8 — Telemetría Distribuida

Fase 2 — Microservicios | Duración: 2 horas | Asistencia: GitHub Copilot

Descripción

Se instrumenta cada microservicio con OpenTelemetry para capturar trazas distribuidas, métricas de latencia y errores. Las trazas permiten ver el recorrido completo de un request a través de todos los servicios.

Temas débiles que cubre

- Transversal: observabilidad es competencia critica en producción
- NodeJS — Performance y Profiling: métricas OpenTelemetry complementan el profiling del M2

Que se implementa

Que implementar	Como	Herramienta
SDK OpenTelemetry	@opentelemetry/sdk-node configurado en cada servicio	OpenTelemetry

Que implementar	Como	Herramienta
Trazas distribuidas	trace.startSpan() en cada operación crítica	OpenTelemetry Tracer
Exportar a Grafana	OTLP exporter apuntando al endpoint gratuito de Grafana	Grafana OTLP
Dashboard básico	Latencia p50/p95, tasa de errores, volúmenes de búsqueda	Grafana
Alertas simples	Alerta si latencia del endpoint supera 500ms	Grafana Alerting
Logs estructurados	Pino en NestJS para logs JSON compatibles con Loki	Pino + Loki

Modulo 9-A — Nube y Cierre

Fase 2 — Microservicios | Duración: 2 horas | Asistencia: GitHub Copilot

Descripción

El módulo final despliega el sistema completo en la nube con algún orquestador. El pipeline CI/CD del módulo 6 se actualiza para incluir el despliegue como paso final. TechDocs queda accesible desde una URL pública.

Temas débiles que cubre

- Angular — Integración continua: pipeline incluye deploy real a la nube
- NodeJS — DevOps, CI/CD y Nube: despliegue en orquestador con variables de entorno de producción

Que se implementa

Que implementar	Como	Herramienta
Deploy en orquestador	Conectar repositorio GitHub, configurar build	A definir
Variables de producción	DATABASE_URL, JWT_SECRET, RABBITMQ_URL en el panel	Secrets cloud
Actualización pipeline CD	Ultimo paso: deploy to orquestador using CLI	GitHub Actions + orquestador
URL de producción	Documentar URL publica y credenciales de demo	Markdown
Revisión de seguridad	Verificar endpoints protegidos con JWT en producción	Postman / curl
Retrospectiva	Revisar temas mejorados, próximos pasos	Reunión de equipo

5. Stack Tecnológico

Herramientas y versiones recomendadas

Capa	Tecnologia	Version	Rol en el proyecto
Backend	NestJS	v10+	API REST, microservicios, seguridad
Frontend	Angular	v17+	UI principal, enrutamiento, librerías
Preview	React	v18+	Panel de preview de documentos
Base de datos	PostgreSQL	v15+	Almacenamiento principal
ORM	TypeORM	v0.3+	Modelado y migraciones
Mensajería	RabbitMQ	v3.12+	Comunicación entre microservicios
Pruebas E2E	Cypress	v13+	Pruebas de flujo completo
Pruebas unit.	Jest	v29+	Tests de servicios y componentes
IA asistente	GitHub Copilot	Actual	Aceleración en todo el proceso
Contenedores	Docker + Compose	Actual	Empaquetado y orquestación
CI/CD	GitHub Actions	Actual	Pipeline de tests y despliegue
Observabilidad	OpenTelemetry + Grafana	Actual	Trazas, métricas y alertas
Nube	Cloud	Actual	Hosting en producción

6. Cronograma Sugerido

Distribución de módulos en 9 semanas de trabajo aprox

El cronograma asume una sesión quincenal de trabajo en equipo de 1 a 1:30 horas. Los módulos M3 y M4 son los más críticos y pueden extenderse a dos sesiones.

Semana	Modulo	Tema principal	Duracion
1	M1	Setup arquitectura nestJS, angular y react(uno de los dos)	1 h
2	M2	CRUD con NestJS, enrutamiento y Guards Angular	2 h
3	M3	Seguridad JWT, interceptores y headers	2 h
4-5	M4	Pruebas unitarias (Jest) y E2E (Cypress)	3 h x2
6	M5	Renderizado OnPush y libreria techdocs-ui	3 h
7	M6	CI/CD con GitHub Actions y Docker	2 h
8	M7	Extraccion microservicio busqueda + RabbitMQ	3 h
9	M8+M9	Telemetria OpenTelemetry y despliegue en nube	2 h + 2 h

7. Próximos Pasos

Acciones inmediatas para el equipo

- Compartir este documento con todos los integrantes que participaran en este mini proyecto.
- Crear el repositorio GitHub con la estructura monorepo (backend/ + frontend/ + services/).
- Coordinar la primera sesión de trabajo para completar el Módulo 1 (2 horas).
- Asignar un responsable por modulo para liderar la sesión y preparar el contexto.

8. Arquitectura del Sistema

Diagramas de referencia: monolito, microservicios y pipeline CI/CD

Esta sección presenta los tres diagramas de arquitectura del proyecto TechDocs. Cada diagrama está construido como tabla editable para facilitar su actualización. El primero muestra la arquitectura monolítica (M1-M5). El segundo muestra la evolución a microservicios (M6-M9) con RabbitMQ y observabilidad. El tercero detalla el pipeline CI/CD con sus dos workflows diferenciados.

Diagrama 1 — Arquitectura monolitica (Fase 1: M1 a M5)

Todo el sistema corre en un único proceso NestJS. Angular y React se comunican con el servidor vía HTTP REST protegido con JWT. Los módulos de autenticación, documentos y búsqueda comparten memoria y acceden a una sola base de datos PostgreSQL. Los servicios transversales (Helmet, Pino, OpenTelemetry, Interceptors) aplican a todo el proceso de forma global.

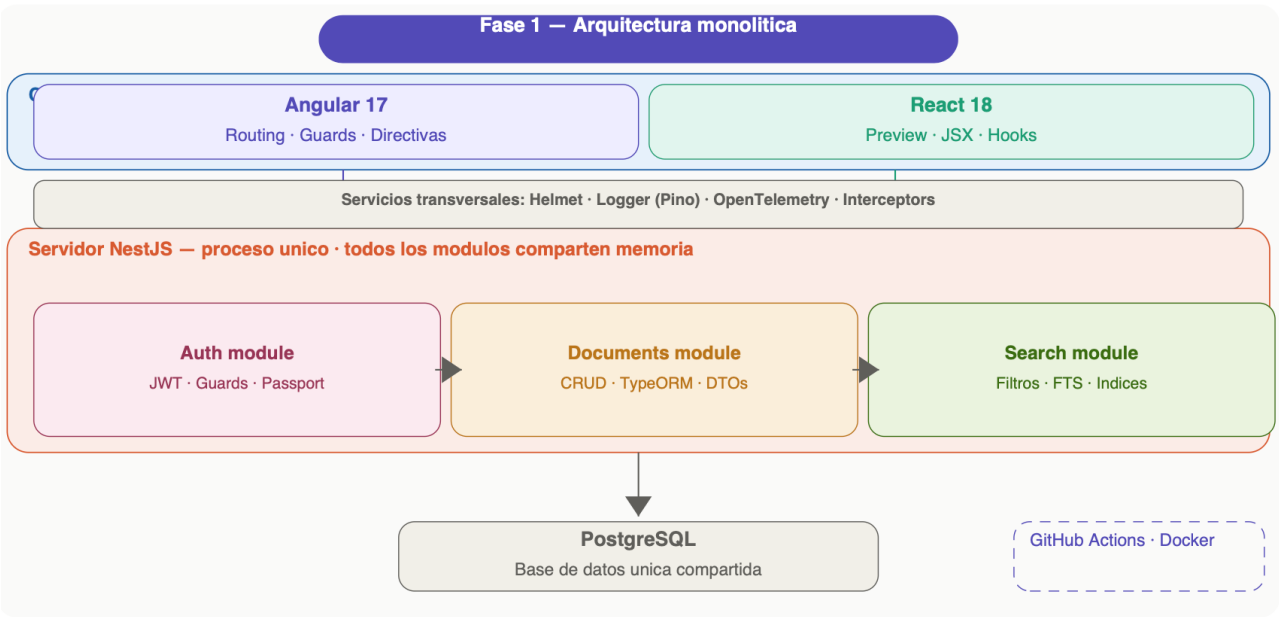


Figura 1 — Monolito TechDocs: un proceso NestJS, una base de datos, despliegue simple con Docker.

CLIENTE — navegador del usuario	
Angular 17	Routing · Guards · Directivas · ChangeDetection · Librerías custom (M5)
React 18	Panel preview de documentos · JSX · Hooks · Virtual DOM
Comunicación con el servidor: HTTP REST autenticado con JWT	
↓ HTTP REST (JWT)	
SERVIDOR NestJS — proceso unico (todos los modulos comparten memoria y dependencias)	
auth-module	JWT · Guards · Passport.js · Login endpoint
documents-module	CRUD · TypeORM · DTOs · Validación con class-validator
search-module	Filtros de texto · FTS · Índices · Consultas avanzadas
—— Servicios transversales aplicados globalmente ——	
Helmet (headers de seguridad) Logger Pino (JSON) OpenTelemetry Interceptors HTTP	

↓ TypeORM — acceso a datos

INFRAESTRUCTURA — Fase 1	
PostgreSQL	Base de datos unica compartida por todos los modulos del servidor
Docker	Un solo Dockerfile + una imagen para todo el proceso NestJS
GitHub Actions	ci.yml (tests en cada PR) + cd.yml (deploy en merge a main)

Figura 1 — Monolito TechDocs: un proceso NestJS, una base de datos compartida, despliegue con un solo contenedor Docker.

Diagrama 2 — Arquitectura de microservicios (Fase 2: M6 a M9)

Cada responsabilidad vive en su propio proceso NestJS independiente. El API Gateway es el único punto de entrada desde el cliente. Los servicios se comunican vía mensajes RabbitMQ (asíncrono) y cada uno posee su propia base de datos. OpenTelemetry captura trazas que cruzan múltiples servicios y las envía a Grafana Cloud. GitHub Actions construye y publica una imagen Docker por servicio.

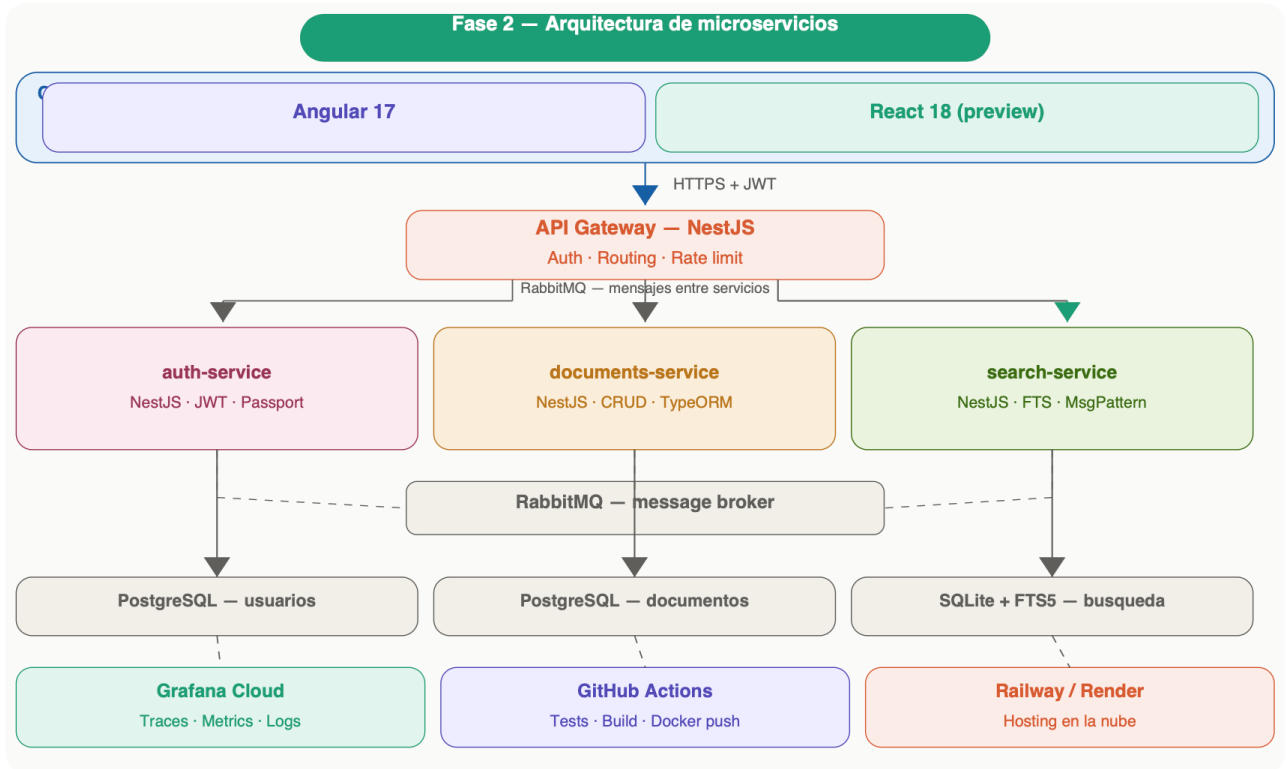


Figura 2 — Microservicios TechDocs: tres servicios independientes, RabbitMQ, bases de datos propias y observabilidad distribuida.

CLIENTE — navegador del usuario		
Angular 17 Routing · Guards · Directivas · Librerías custom · ChangeDetection OnPush		
React 18 Panel preview de documentos · JSX · Hooks		
Comunicación: HTTPS + JWT únicamente hacia el API Gateway		
↓ HTTPS + JWT		
API GATEWAY — NestJS (unico punto de entrada desde el exterior)		
Autenticación JWT Enrutamiento de requests hacia el servicio correcto		
Rate limiting Validación de tokens antes de reenviar al servicio destino		
↓ RabbitMQ — message broker entre servicios (mensajes asincronos)		
auth-service NestJS · JWT · Passport Login endpoint JwtAuthGuard Token generacion BD: PostgreSQL (usuarios y sesiones)	documents-service NestJS · CRUD · TypeORM Crear/editar docs Versionado Validacion DTOs BD: PostgreSQL (documentos y versiones)	search-service NestJS · FTS · MessagePattern Busqueda full-text Indice FTS5 Responde a RabbitMQ BD: SQLite + FTS5 (indice de busqueda)

INFRAESTRUCTURA — Fase 2	
RabbitMQ	Message broker — los servicios publican y consumen mensajes de forma asíncrona
OpenTelemetry	Instrumentación en cada servicio → trazas distribuidas cross-service
Grafana Cloud	Dashboard: latencia p50/p95 · tasa de errores · volúmenes · alertas
GitHub Actions	Build de imagen Docker por servicio + push a ghcr.io + deploy a Railway
Cloud	Hosting en la nube — URL publica de producción con health checks

Figura 2 — Microservicios TechDocs: tres servicios independientes, RabbitMQ, bases de datos propias y observabilidad distribuida.

Diagrama 3 — Pipeline CI/CD con GitHub Actions y Docker

El pipeline tiene dos workflows completamente separados. ci.yml protege la rama main ejecutandose en cada Pull Request: instala dependencias con cache, corre los tests unitarios de NestJS (jest --coverage), los tests de Angular en modo headless y el linter. Si falla, el PR queda bloqueado. cd.yml solo se activa al hacer merge a main: construye el binario de produccion, genera la imagen Docker con multi-stage build (sin devDependencies), la publica en GitHub Container Registry y despliega en un orquestador. Un health check final verifica que el servicio responde antes de dar el pipeline por exitoso.

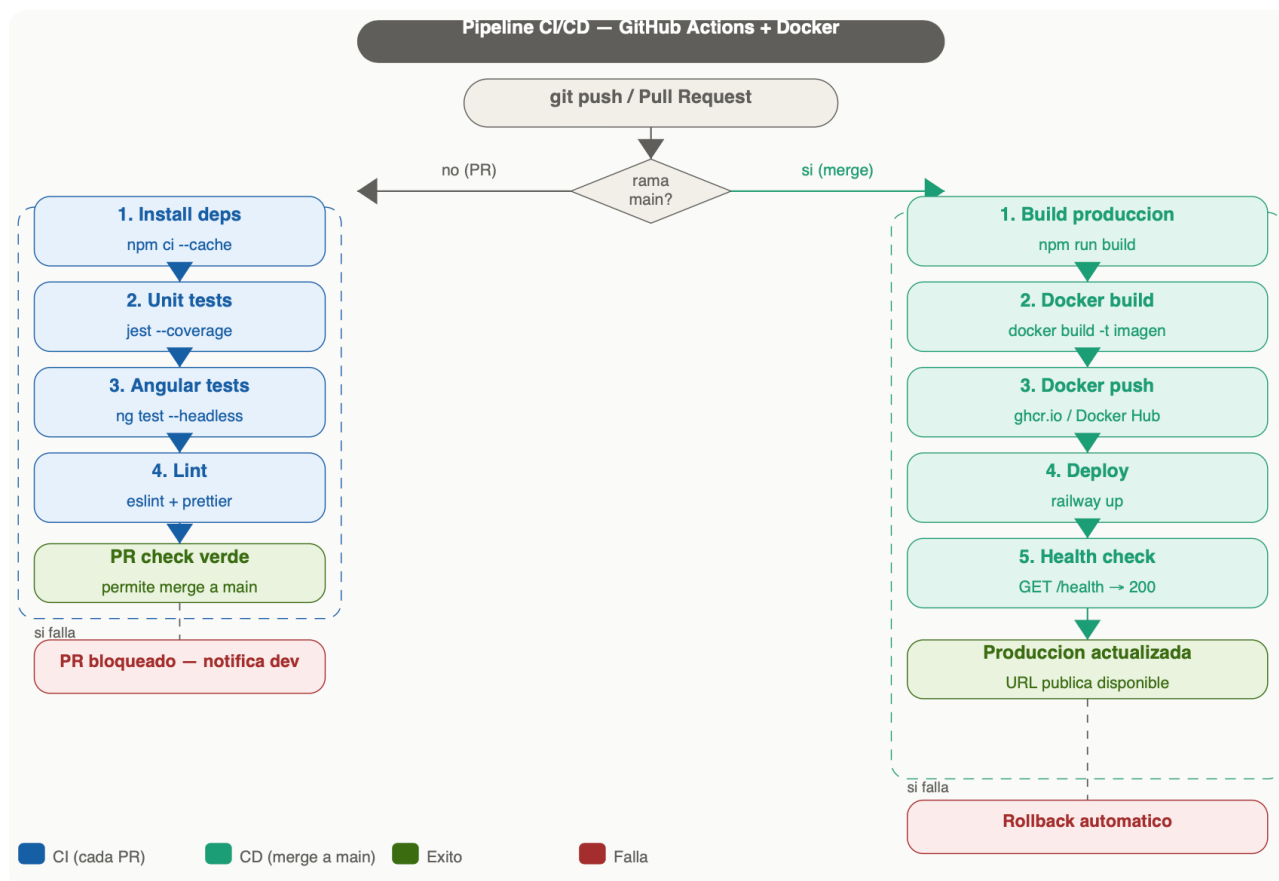


Figura 3 — Pipeline CI/CD: ci.yml por Pull Request (izquierda) y cd.yml por merge a main (derecha).

TRIGGER — evento en GitHub

git push a cualquier rama → evalua la condicion de rama

Pull Request hacia main → activa ci.yml

Workflow: ci.yml (cada Pull Request)

1. npm ci --cache Instalar dependencias con cache de npm
2. jest --coverage Tests unitarios NestJS (cobertura > 70%)
3. ng test --headless Tests de componentes Angular (TestBed)
4. eslint + prettier check Lint y formato de codigo

Exito: PR check verde — permite hacer merge a main

Falla: PR bloqueado — notificación por email al desarrollador

Merge / push a main → activa cd.yml

Workflow: cd.yml (merge a main)

1. npm run build Compilar TypeScript a dist/ (produccion)
2. docker build --target runner Multi-stage: solo binario + node_modules
3. docker push ghcr.io/... Publicar imagen en GitHub Container Registry
4. cloud --service X Desplegar nueva imagen en el entorno de produccion
5. curl /health → 200 OK Verificar que el servicio responde correctamente

Exito: Produccion actualizada — URL publica disponible

Falla: Rollback automatico — cloud activa la version anterior

Figura 3 — Pipeline CI/CD: ci.yml ejecuta tests en cada PR (izquierda); cd.yml construye, publica y despliega en cada merge a main (derecha).

Comparación de ambas arquitecturas

Aspecto	Monolito (M1-M5)	Microservicios (M6-M9)
Procesos en ejecución	1 proceso único	3+ procesos independientes
Base de datos	1 PostgreSQL compartida	1 BD propia por servicio
Comunicación interna	Llamadas directas en memoria	RabbitMQ — mensajes asíncronos
Si un servicio falla	Cae todo el sistema	Solo ese servicio se ve afectado
Tiempo de build/deploy	~1 minuto (un contenedor)	~3-4 min (paralelo por servicio)
Complejidad operacional	Baja — un solo docker run	Media — Docker Compose + broker
Observabilidad	Logs locales con Pino	OpenTelemetry + Grafana Cloud
Cuando es la opción correcta	Inicio del proyecto, equipos chicos	Escala, equipos y dominios separados